

■ 2.5.3 Conditionals

In building procedures in *Mathematica*, you often need to specify that a set of operations is to be performed only if certain conditions are satisfied. *Mathematica* provides a number of “conditional structures” that allow you to do this.

<code>If[test, then, else]</code>	execute <i>then</i> if <i>test</i> is True , and <i>else</i> if it is False
<code>Which[test₁, value₁, test₂, ...]</code>	evaluate <i>test</i> ₁ , ... in turn, giving the value associated with the first test that is True
<code>Switch[expr, form₁, value₁, ...]</code>	compare <i>expr</i> with <i>form</i> ₁ , <i>form</i> ₂ , ..., giving the value associated with the first form that it matches

Basic conditional constructs.

The test gives **False**, and so the “*else*” expression *y* is returned.

```
In[1]:= If[7 > 8, x, y]
Out[1]= y
```

This defines a step function, with value 1 for $x > 0$, and -1 otherwise.

```
In[2]:= f[x_] := If[x > 0, 1, -1]
```

Here is another definition of the step function, using transformation rules with conditions attached. This kind of approach is often much better in *Mathematica* than using an explicit **If** statement.

```
In[3]:= g[x_] := 1 /; x > 0 ; g[x_] := -1 /; x <= 0
```

This defines a function with three regions. Using **True** as the third test makes this the default case.

```
In[4]:= h[x_] := Which[x < 0, x^2, x > 5, x^3, True, 0]
```

This uses the first case in the **Which**.

```
In[5]:= h[-5]
Out[5]= 25
```

This uses the third case.

```
In[6]:= h[x^2]
Out[6]= 0
```

This defines a function that depends on the value of its argument modulo 3.

```
In[7]:= r[x_] := Switch[Mod[x, 3], 0, a, 1, b, 2, c]
```

Mod[7, 3] is 1, so this uses the second case in the **Switch**.

```
In[8]:= r[7]
Out[8]= b
```

The forms you use in `Switch` can be any patterns. The pattern `_` matches any expression, so that in this case the last choice in the `Switch` is used if none of the others apply.

```
In[9]:= Switch[Mod[x, 3], 0, a, 1, b, _, q]
Out[9]= q
```

One subtlety of conditionals in a symbolic system like *Mathematica* is that the conditions you specify may give neither `True` nor `False`. For example, `x == y` does not yield a definite truth value unless `x` and `y` have been given special values (such as numerical ones). As a result, the *Mathematica* `If` structure has a fourth element, which you can use to specify the result when the truth value of the condition is unknown.

<code>If[test, then, else, unknown]</code>	the full form of <code>If</code> , including the result to give if <i>test</i> gives neither <code>True</code> nor <code>False</code>
<code>TrueQ[expr]</code>	give <code>True</code> if <i>expr</i> is <code>True</code> , and <code>False</code> otherwise
<code>SameQ[expr₁, expr₂]</code>	give <code>True</code> if two expressions are identical, and <code>False</code> otherwise

Functions for dealing with symbolic conditions.